Floating Point Literals :

* If a numeric literal contains decimal OR fraction then it is called Floating Point Literals.
  Example :
  12.90, 6.9, 0.1

* In floating point literals we have two data types :
  a) float (32 bits)
  b) double (64 bits)

* By default, every floating point literal is of type **double only** so, If we write the following statement we will get compilation error :

```
float f = 0.0;  //error
```

Converting double to float :

```
float f1 = 0.0F;
float f2 = 0.4f;
float f3 = (float) 1.2;
```

* Even though, all the floating point literals are by default of type **double only** but still java compiler has provided two flavours to represnt the double value explicitly to **enhance the redability of the code.**

  Example :

```
double d1 = 12D;
double d2 = 15d;
```

* Floating point literals we can represent in **exponent form.**

  Example :

  double d1 = 15e2;

  $d1 = 15 \times 10^{2}$;

  = 1500.0

  double d2 = 15^{-3};

  d2 = 0.015

* While working with integral literal, we can represent integral literal in four different forms decimal, octal, hexadecimal and binary but floating point literals, we can represent in only **one form i.e decimal only**

* Integral literal data type (byte, short, int and long) we can assign to floating point literal (float & double) but floating point literal, we cannot assign directly to integer literal otherwise there is a chance of loss without explicit type casting.

  Example :

```
float f = 12.9;
int x = f; // by default not possible otherwise int will hold only 12
```

//Programs on floating point literals :

```
void main()
{
     float f = 0.0; //error
     IO.println(f);
}


void main()
{
     float b = 15.29F;
     float c = 15.25f;
     float d = (float) 15.30;
     IO.println(b +" : "+c+" : "+d);
}


void main()
{
     double d = 15.15;
     double e = 15d;
     double f = 90D;
     IO.println(d+" , "+e+" , "+f);
}

void main()
{
     double x = 0129.89;

     double y = 0167;

     double z = 0178; //error

     IO.println(x+","+y+","+z);
}

void main()
{
     double x = 0X29;

     double y = 0X91.5; //error

     double z = 0B0.01; //error

     IO.println(x+","+y+", "+z);
}



void main()
{
     double d1 = 15e-3;
     IO.println("d1 value is :"+d1);

     double d2 = 15e3;
     IO.println("d2 value is :"+d2);
}


void main()
{
     double a = 0791; //error
     double b = 0791.0;
     double c = 0777;
     double d = 0Xdead;
     double e = 0Xdead.0; //error
}

void main()
{
        double a = 1.5e3;
        float b = 1.5e3; //Error
        float c = 1.5e3F;
        double d = 10;
        int e = 10.5; //error
        long f = 10D; //error
        int g = 10F;  //error
        long l = 12.78F; //error
}

//Range and size of floating point literal
void main()
{
     IO.println("\n Float range:");
     IO.println(" min: " + Float.MIN_VALUE);
     IO.println(" max: " + Float.MAX_VALUE);
     IO.println(" size :"+Float.SIZE);

     IO.println("\n Double range:");
     IO.println(" min: " + Double.MIN_VALUE);
     IO.println(" max: " + Double.MAX_VALUE);
     IO.println(" size :"+Double.SIZE);
}
```

By using System.out.printf() method we can print the decimal numbers according to our format. Here we should use , to separate two parameters.

```
void main()
{
  double d1 = 10.0/3.0;

  //IO.println(d1); //3.3333333333333335

  //print only 2 digits after the decimal
  System.out.printf("Value is %.2f ", d1);

}
```